Plan: Domain Registration Feature — WebsiteExperts

Plan date: 2026-05-29
Implementation stage: MVP (manual) → API Integration → Bundling
Stack: Laravel 13 + Inertia.js/React + Filament 5 + Stripe

Context and Approach

Selling domains as an **entry product** leading to website, hosting, email, SSL and maintenance.
We are not building a full GoDaddy. We are building:

1. First a manual MVP (form + order + Stripe + manual registration at provider)
2. Then automation via API (OpenSRS or Openprovider)
3. Finally bundling with other platform products

STAGE 1 — MVP (manual / semi-automatic)

1.1 Database — new migrations

Table creation order:

```
domain_orders         — domain orders
domains               — registered domains
domain_contacts       — contact data for registration (WHOIS)
domain_price_list     — price list (snapshot for quoting purposes)
domain_renewals       — renewal schedule
domain_events         — domain event log (registration, transfer, renewal)
```

Schema `domain_orders`:

```
id, business_id, user_id, domain_name, tld, years,
action (register|transfer|renew),
status (pending_payment|paid|registering|completed|failed|cancelled),
provider, wholesale_price, retail_price, currency,
payment_id, stripe_payment_intent_id,
notes, admin_notes,
timestamps
```

Schema `domains`:

```
id, business_id, user_id, domain_order_id,
provider, provider_domain_id,
name, tld, full_domain,
```

```
status (pending|active|expired|transferred|cancelled),
registered_at, expires_at, auto_renew (bool),
nameservers (json), dns_records (json),
whois_privacy (bool),
timestamps
```

Schema `domain_price_list`:

```
id, tld, register_price, renew_price, transfer_price,
currency, is_active, notes,
timestamps
```

Schema `domain_renewals`:

```
id, domain_id, due_date, years, status (pending|completed|failed),
notified_30d, notified_14d, notified_7d, notified_1d,
timestamps
```

1.2 Laravel Models

- `DomainOrder` — relations: `belongsTo(Business)`, `belongsTo(User)`, `hasOne(Domain)`
- `Domain` — relations: `belongsTo(Business)`, `belongsTo(User)`, `belongsTo(DomainOrder)`, `hasMany(DomainRenewal)`, `hasMany(DomainEvent)`
- `DomainPriceList`
- `DomainRenewal`
- `DomainEvent`
- `DomainContact`

1.3 Service Layer (contract + implementations)

Interface:

```php
// app/Services/Domain/DomainRegistrarInterface.php
interface DomainRegistrarInterface
{
    public function search(string $query): array;
    public function checkAvailability(string $domain): DomainAvailabilityResult;
    public function register(DomainRegistrationPayload $payload):
DomainRegistrationResult;
    public function renew(string $domain, int $years): DomainRenewalResult;
    public function updateNameservers(string $domain, array $nameservers): bool;
    public function getDomainInfo(string $domain): DomainInfoResult;
    public function transfer(string $domain, string $authCode):
```

```
DomainTransferResult;
}
```

Data Transfer Objects (DTOs):

```
app/Data/Domain/
  DomainAvailabilityResult.php
  DomainRegistrationPayload.php
  DomainRegistrationResult.php
  DomainRenewalResult.php
  DomainInfoResult.php
  DomainTransferResult.php
  DomainSearchResult.php
  DomainPriceSnapshot.php
```

Service implementations:

```
app/Services/Domain/
  ManualDomainRegistrarService.php     ← MVP: manual handling (stub without API)
  OpenSrsRegistrarService.php          ← Stage 3
  OpenProviderRegistrarService.php     ← Stage 3
  DomainPricingService.php             ← price list management
  DomainRenewalService.php             ← sending renewal notifications
  DomainOrderService.php               ← order logic
```

Service Provider binding (DomainServiceProvider):

```
// In stage 1 binds ManualDomainRegistrarService
// In stage 3 switch to OpenSrsRegistrarService
$this->app->bind(DomainRegistrarInterface::class,
ManualDomainRegistrarService::class);
```

1.4 Actions

```
app/Actions/Domain/
  CreateDomainOrderAction.php
  ProcessDomainPaymentAction.php
  RegisterDomainAction.php
  RenewDomainAction.php
  CancelDomainOrderAction.php
  UpdateNameserversAction.php
  CheckDomainAvailabilityAction.php
  SendRenewalReminderAction.php
```

1.5 Jobs

```
app/Jobs/
  RegisterDomainJob.php                ← fires after successful payment
  SendDomainRenewalReminderJob.php
  CheckDomainExpiryJob.php             ← scheduled cron
```

1.6 Controllers

```
app/Http/Controllers/
  Domain/
    PublicDomainController.php         ← public page + search
    DomainOrderController.php          ← order flow (checkout)
  Portal/
    DomainController.php               ← client panel: domain list, renewal
    DomainCheckoutController.php       ← Stripe payment for domain
```

1.7 Routes

Public (routes/web.php):

```
// Public domain registration
Route::get('/domains', [PublicDomainController::class, 'index'])-
>name('domains.index');
Route::get('/domains/check', [PublicDomainController::class, 'check'])-
>name('domains.check');
Route::post('/domains/check', [PublicDomainController::class, 'checkPost'])-
>name('domains.check.post');
```

Client portal (/portal/domains):

```
Route::prefix('domains')->name('portal.domains.')->group(function () {
    Route::get('/', [DomainController::class, 'index'])->name('index');
    Route::get('/order', [DomainController::class, 'order'])->name('order');
    Route::post('/order', [DomainOrderController::class, 'store'])-
>name('order.store');
    Route::get('/order/{order}/checkout', [DomainCheckoutController::class,
'show'])->name('checkout');
    Route::post('/order/{order}/pay', [DomainCheckoutController::class, 'pay'])-
>name('pay');
    Route::get('/{domain}', [DomainController::class, 'show'])->name('show');
    Route::post('/{domain}/renew', [DomainController::class, 'renew'])-
>name('renew');
```

```
    Route::patch('/{domain}/nameservers', [DomainController::class,
'updateNameservers'])->name('nameservers.update');
});
```

1.8 Notifications

```
app/Notifications/
  DomainOrderPlacedNotification.php        ← to client after ordering
  DomainRegisteredNotification.php         ← to client after registration
  DomainExpiryReminderNotification.php     ← 30/14/7/1 days before expiry
  DomainOrderAdminNotification.php         ← to admin about new order
  DomainRegistrationFailedNotification.php
```

STAGE 2 — Admin Panel (Filament)

2.1 Filament Resources

```
app/Filament/Resources/
  DomainOrderResource.php          ← order management
    Pages/
      ListDomainOrders.php
      ViewDomainOrder.php           ← details + actions (approve, cancel, mark as
registered)
  DomainResource.php               ← list of registered domains
    Pages/
      ListDomains.php
      ViewDomain.php
  DomainPriceListResource.php      ← TLD price list editing
    Pages/
      ListDomainPriceLists.php
      EditDomainPriceList.php
```

2.2 Filament Pages / Widgets

```
app/Filament/Widgets/
  DomainOrderStatsWidget.php       ← number of orders, status, domain revenue
  DomainExpiryWidget.php           ← domains expiring within 30 days
```

2.3 Key actions in the admin panel

In `DomainOrderResource` → `ViewDomainOrder`:

- **"Mark as Registered"** — change order status and create `Domain` record

- **"Cancel Order"** — cancel + refund (Stripe refund action)
- **"Add notes"** — admin notes
- **Related domains table** — list of client's domains

In `DomainResource` → `ViewDomain`:

- Nameserver editing
- "Send renewal reminder" button
- Event history (DomainEvent)
- Link to order, to client

STAGE 3 — Public Frontend (Inertia.js/React)

3.1 New React pages

```
resources/js/Pages/
  Domains/
    Index.tsx              ← landing page "Domain Registration UK"
    Check.tsx              ← domain search results
    Order.tsx              ← order summary
    Checkout.tsx           ← Stripe payment
```

3.2 React Components

```
resources/js/Components/
  Domain/
    DomainSearchForm.tsx          ← input + search button
    DomainAvailabilityResult.tsx  ← availability check result
    DomainPriceCard.tsx           ← card with price and TLD
    DomainOrderSummary.tsx        ← order summary
    DomainTldSelector.tsx         ← TLD selector (.co.uk, .com, .uk, .net, .org)
    DomainFeatureList.tsx         ← "What you get with a domain"
    DomainFaq.tsx                 ← FAQ section
    DomainBundleCards.tsx         ← bundle package cards
```

3.3 `/domains` page structure (Index.tsx)

```
[Hero Section]
  "Register your UK business domain"
  DomainSearchForm search widget
  "from £X/year"

[Popular TLDs Section]
  Cards: .co.uk | .uk | .com | .net | .org
  Registration + renewal price
```

```
[Bundles Section]
  1. Domain Registration — from £X/year
  2. Domain + Website Setup — from £X
  3. Domain + Business Email — from £X
  4. Website Launch Package — from £X (domain + hosting + SSL + email + SEO)

[Features Section]
  ✓ Free DNS management
  ✓ WHOIS privacy
  ✓ Auto-renewal reminders
  ✓ Managed for you

[FAQ Section]

[CTA Section]
  "Let us handle everything"
  → Contact form / order
```

3.4 Client portal — new pages

```
resources/js/Pages/Portal/
  Domains/
    Index.tsx         ← list of client's domains (status, expiry date,
nameservers)
    Show.tsx          ← domain details
    Order.tsx         ← order a new domain
    Checkout.tsx      ← domain payment
```

3.5 Portal — widgets / navigation

- Add "Domains" to client portal nav (next to Projects, Invoices, Contracts)
- Add widget to portal dashboard: "Your domains" with expiry alert
- Badge with number of domains expiring within 30 days

STAGE 4 — API Integration (Stage 3 per roadmap)

4.1 Provider selection (recommendation: OpenSRS or Openprovider)

OpenSRS:

- Proven reseller platform
- XML + REST API
- White-label
- WHMCS modules (optional)
- Packages: domains + email + SSL

Openprovider:

- 1900+ TLDs
- REST API
- WHMCS modules
- Good for .uk + Nominet

4.2 `OpenSrsRegistrarService` implementation

```
// app/Services/Domain/OpenSrsRegistrarService.php
class OpenSrsRegistrarService implements DomainRegistrarInterface
{
    public function __construct(
        private readonly OpenSrsClient $client,
        private readonly DomainEventRepository $events,
    ) {}

    public function checkAvailability(string $domain): DomainAvailabilityResult {
... }
    public function register(DomainRegistrationPayload $payload):
DomainRegistrationResult { ... }
    public function renew(string $domain, int $years): DomainRenewalResult { ... }
    // etc.
}
```

4.3 Binding change in DomainServiceProvider

```
// config/services.php — add domain_registrar section
// .env — DOMAIN_REGISTRAR_PROVIDER=opensrs

$this->app->bind(DomainRegistrarInterface::class, function ($app) {
    return match(config('services.domain_registrar.provider')) {
        'opensrs'       => $app->make(OpenSrsRegistrarService::class),
        'openprovider'  => $app->make(OpenProviderRegistrarService::class),
        default         => $app->make(ManualDomainRegistrarService::class),
    };
});
```

4.4 Handling webhooks from the provider

```
app/Http/Controllers/Domain/
  DomainProviderWebhookController.php   ← events from OpenSRS/Openprovider
```

```
// routes/web.php (CSRF exempt)
Route::post('/webhooks/domain/{provider}',
DomainProviderWebhookController::class);
```

STAGE 5 — Bundling with other products

5.1 Sales packages

Package	Contents	Price
Domain Only	Domain + DNS + WHOIS privacy	from £12/year
Domain + Email	Domain + business email	from £25/year
Domain + Website	Domain + DNS setup + connection	from £X
Website Launch	Domain + website + hosting + SSL + email + SEO	from £X

5.2 Integration with existing modules

- **Invoice** → after domain registration automatically generate an invoice
- **Client** → domains assigned to client in CRM
- **Project** → ability to associate a domain with a project
- **Sales Offer / Quote** → add domain as a line item in offer/quote
- **Automation** → rule: "30 days before expiry → send notification"

5.3 Integration with price calculator

- Add "Domain Registration" as a step in `CalculatorStep`
- Ability to add domain to project quote

STAGE 6 — Legal and Formal Aspects

6.1 Required documents (before selling)

- [ ] Domain registration terms of service
- [ ] Renewal policy (deadlines, automatic renewal)
- [ ] Cancellation and refund policy
- [ ] Domain transfer policy
- [ ] Privacy Policy update (GDPR)
- [ ] Information about upstream registrar (OpenSRS/Openprovider)
- [ ] DNS abuse reporting procedure
- [ ] Price list with clear distinction between registration and renewal prices

6.2 New CMS pages (`/p/slug`)

- `/p/domain-registration-terms` — Domain Registration Terms of Service
- `/p/domain-renewal-policy` — Renewal Policy
- `/p/domain-transfer-policy` — Transfer Policy
- `/p/dns-abuse-policy` — DNS Abuse Policy

Implementation Order (sprint plan)

Sprint 1 — Foundation

1. Migrations: `domain_price_list`, `domain_orders`, `domains`, `domain_renewals`, `domain_events`
2. Models with relations
3. `DomainRegistrarInterface` + DTOs
4. `ManualDomainRegistrarService` (stub)
5. `DomainPricingService`

Sprint 2 — Admin panel (Filament)

1. `DomainPriceListResource` + price list editing
2. `DomainOrderResource` with details view + actions (approve, cancel)
3. `DomainResource`
4. Dashboard widgets: stats + expiry

Sprint 3 — Public frontend

1. `/domains` page (React + Inertia)
2. Domain search (form → check → results)
3. Order flow: Order → Checkout → Stripe payment
4. Order confirmation + email

Sprint 4 — Client portal

1. `/portal/domains` page — client's domain list
2. Domain details (`/portal/domains/{domain}`)
3. Domain renewal + payment
4. Portal dashboard widget
5. Portal navigation — adding "Domains"

Sprint 5 — Notifications and cron

1. Email notifications (order, registration, expiry)
2. `CheckDomainExpiryJob` (scheduled)
3. `SendDomainRenewalReminderJob`
4. `RegisterDomainJob`

Sprint 6 — API Integration (after first clients)

1. Reseller account registration (OpenSRS or Openprovider)
2. `OpenSrsRegistrarService` or `OpenProviderRegistrarService`
3. Sandbox tests
4. Binding change in DomainServiceProvider
5. Webhook handler from provider

Sprint 7 — Bundling

1. Domain association with Invoice, Project, Quote, SalesOffer
2. "Domain" step in calculator
3. Sales packages on `/domains` page

4. Legal documents (CMS pages)

Key Architectural Decisions

Decision	Choice	Rationale
Payment	Stripe (existing)	Already integrated in the platform
Provider API (Stage 3)	OpenSRS or Openprovider	Wide TLD coverage, REST API, white-label
Registration flow	Pay → Job → Register	Avoids losing domain between search and payment
Service architecture	Interface + adapters	Swap provider without changes in controllers
MVP	ManualDomainRegistrarService	Zero API integration at start
Premium domain prices	Explicit acknowledgement	In accordance with ICANN and best practices

---

Files to Create (full list)

Database

```
database/migrations/
  xxxx_create_domain_price_list_table.php
  xxxx_create_domain_orders_table.php
  xxxx_create_domains_table.php
  xxxx_create_domain_contacts_table.php
  xxxx_create_domain_renewals_table.php
  xxxx_create_domain_events_table.php
```

Backend (PHP)

```
app/
  Data/Domain/
    DomainAvailabilityResult.php
    DomainRegistrationPayload.php
    DomainRegistrationResult.php
    DomainRenewalResult.php
    DomainInfoResult.php
    DomainTransferResult.php
    DomainSearchResult.php
    DomainPriceSnapshot.php
  Models/
    Domain.php
    DomainOrder.php
    DomainContact.php
    DomainPriceList.php
```

```
    DomainRenewal.php
    DomainEvent.php
  Services/Domain/
    DomainRegistrarInterface.php
    ManualDomainRegistrarService.php
    OpenSrsRegistrarService.php        (Stage 3)
    OpenProviderRegistrarService.php   (Stage 3)
    DomainPricingService.php
    DomainRenewalService.php
    DomainOrderService.php
  Actions/Domain/
    CreateDomainOrderAction.php
    ProcessDomainPaymentAction.php
    RegisterDomainAction.php
    RenewDomainAction.php
    CancelDomainOrderAction.php
    UpdateNameserversAction.php
    CheckDomainAvailabilityAction.php
    SendRenewalReminderAction.php
  Http/Controllers/
    Domain/
      PublicDomainController.php
      DomainOrderController.php
      DomainProviderWebhookController.php
    Portal/
      DomainController.php
      DomainCheckoutController.php
  Jobs/
    RegisterDomainJob.php
    SendDomainRenewalReminderJob.php
    CheckDomainExpiryJob.php
  Notifications/
    DomainOrderPlacedNotification.php
    DomainRegisteredNotification.php
    DomainExpiryReminderNotification.php
    DomainOrderAdminNotification.php
    DomainRegistrationFailedNotification.php
  Filament/Resources/
    DomainOrderResource.php
    DomainResource.php
    DomainPriceListResource.php
  Filament/Widgets/
    DomainOrderStatsWidget.php
    DomainExpiryWidget.php
  Providers/
    DomainServiceProvider.php
```

Frontend (React/TypeScript)

```
resources/js/
  Pages/
```

```
    Domains/
      Index.tsx
      Check.tsx
      Order.tsx
      Checkout.tsx
    Portal/Domains/
      Index.tsx
      Show.tsx
      Order.tsx
      Checkout.tsx
  Components/Domain/
    DomainSearchForm.tsx
    DomainAvailabilityResult.tsx
    DomainPriceCard.tsx
    DomainOrderSummary.tsx
    DomainTldSelector.tsx
    DomainFeatureList.tsx
    DomainFaq.tsx
    DomainBundleCards.tsx
    DomainStatusBadge.tsx
    DomainExpiryAlert.tsx
```

Translations

```
lang/
  en/domain.php
  pl/domain.php
  pt/domain.php
```

Notes / To Be Decided

- [ ] Which TLDs do we support at launch? Recommendation: .co.uk, .uk, .com, .net, .org
- [ ] Starting prices — to be determined after verifying wholesale prices with provider
- [ ] Should WHOIS privacy be included in the price or as an add-on?
- [ ] Can domains be ordered independently, or only as part of a project/website?
- [ ] Minimum registration period: 1 year
- [ ] Auto-renewal: disabled by default (client decides)
- [ ] Provider for Stage 3 integration: OpenSRS vs Openprovider — decision pending